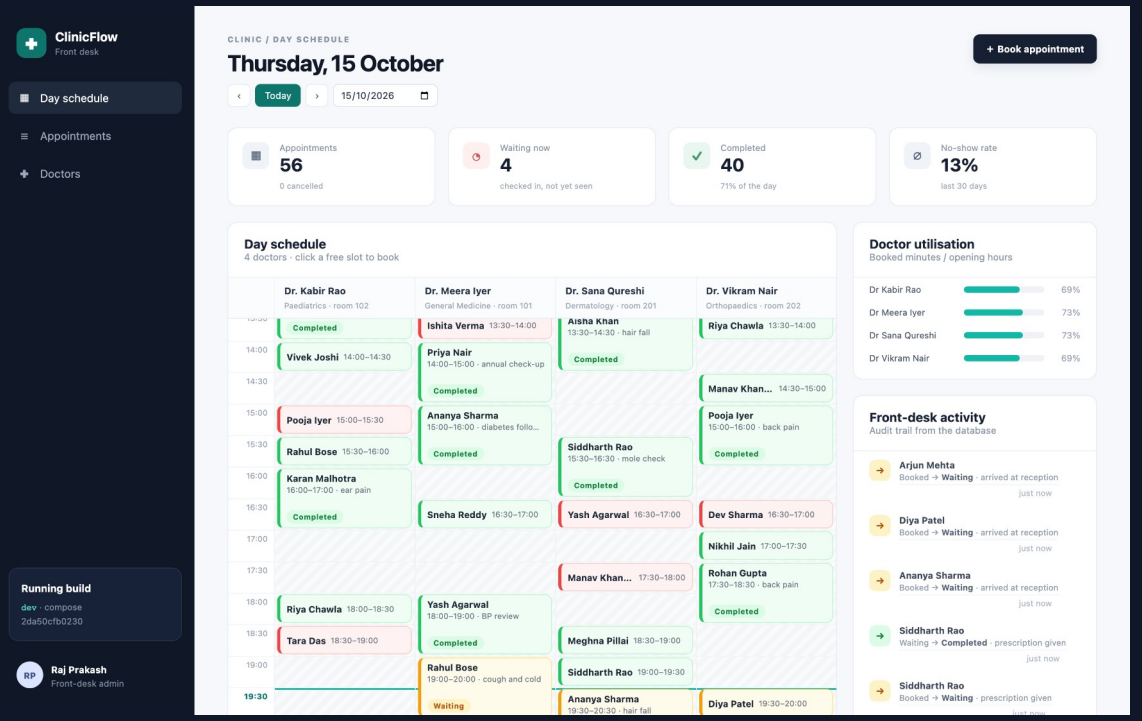
ClinicFlow

From a laptop to a monitored Kubernetes cluster: a DevSecOps capstone



Raj Prakash · 24BCS10328 · Session 21 final project

github.com/RajPrakash681/devsecops-python

A front desk for a multi-doctor clinic

Book without double-booking

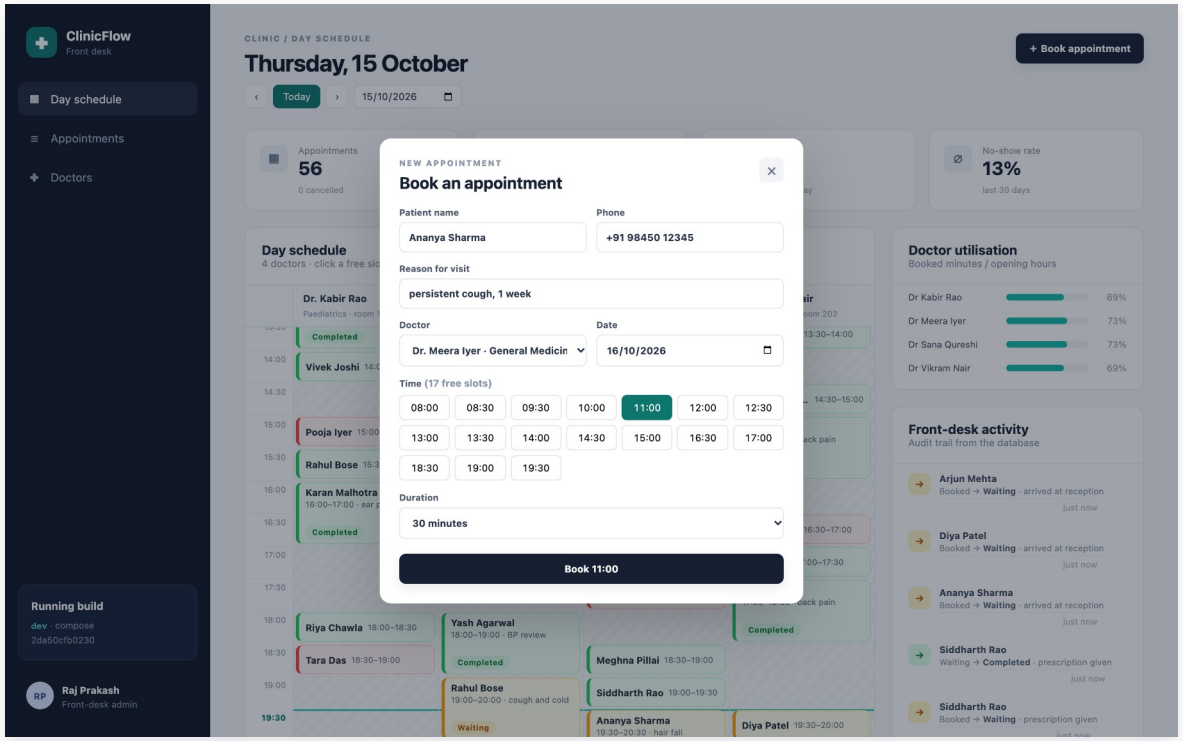
Free slots per doctor; overlaps rejected under a row lock (409)

Check-in workflow with an audit trail

Booked → Waiting → Completed, or Cancelled / No-show; illegal moves refused

Live clinic KPIs

Waiting room, doctor utilisation and the 30-day no-show rate



Domain is my own; the DevOps layer follows the course's TaskBoard architecture.

Tech stack



Application

React 19 · FastAPI · PostgreSQL 17



Testing

pytest · 32 cases · 92% coverage



Containers

Multi-stage builds · non-root



CI/CD

GitHub Actions · GHCR · SHA tags



Security

Trivy · Gitleaks · Bandit · PSS



Infrastructure

Terraform · AWS VPC + EKS



Platform

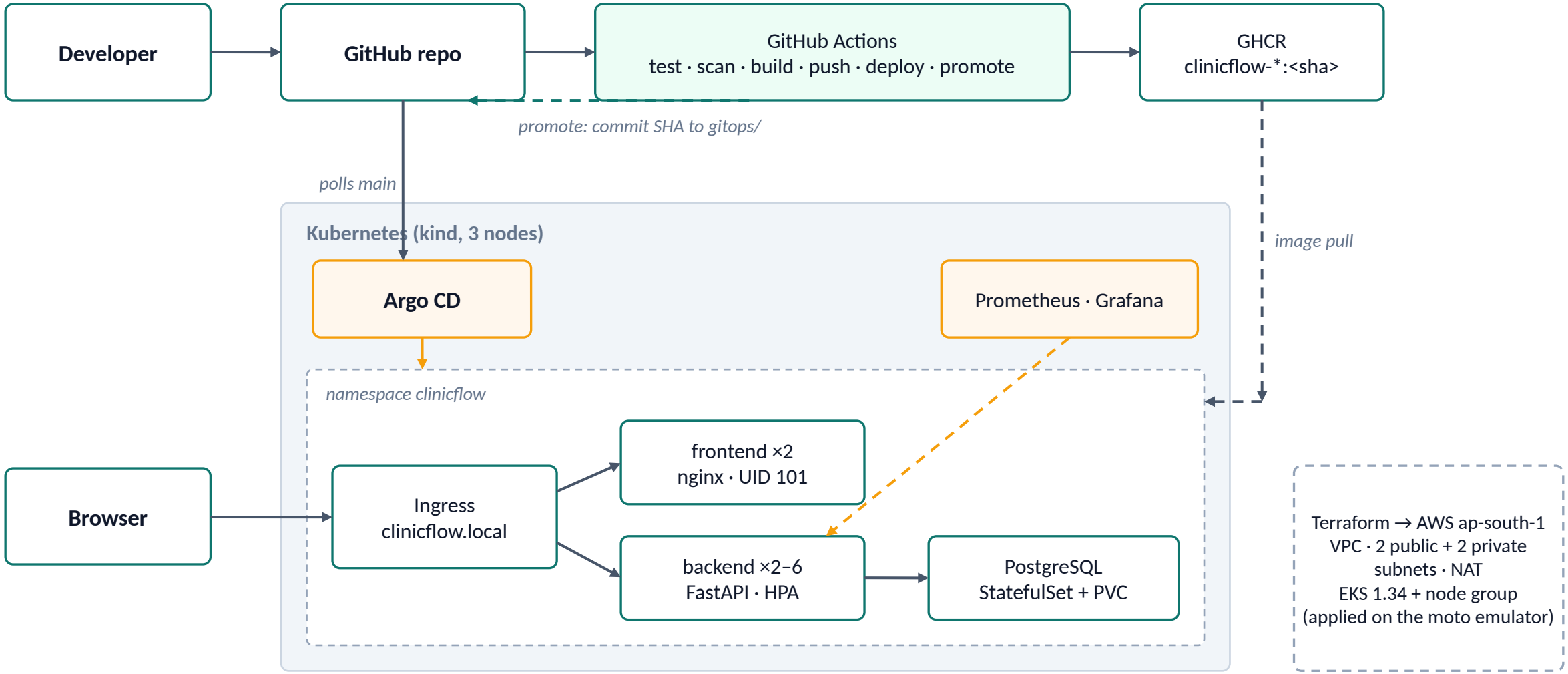
Kubernetes · Helm · Argo CD



Observability

Prometheus · Grafana · alerts

Architecture

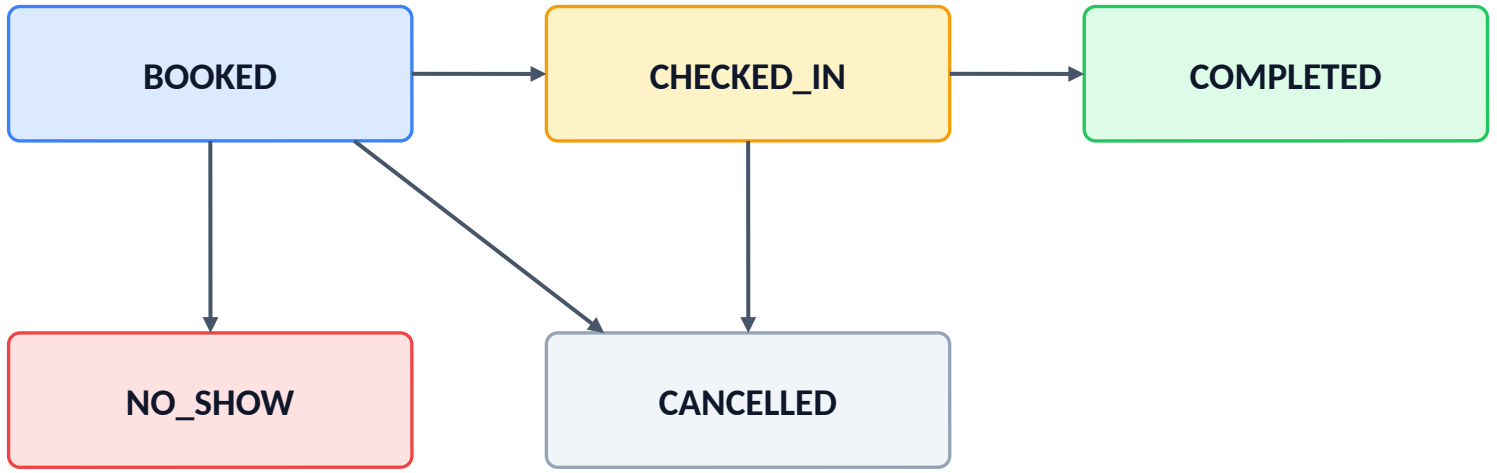


Application: correctness lives in the database too

17
endpoints (14 under /api)

3
Alembic migrations

32
pytest cases on a throwaway DB



Any other move → 409, naming the allowed ones. Every change is written to appointment_events.

No double booking
Booking locks the doctor's row (SELECT … FOR UPDATE) and checks for overlap, so two replicas can't both take the same slot.

Database guard rails
CHECK (ends_at > starts_at), a status CHECK, RESTRICT on doctors with history, CASCADE for the audit trail.

Safe migrations
A Postgres advisory lock makes replicas migrate one at a time.

CI/CD: eleven jobs from push to production



pytest · migrations race
frontend · Gitleaks
Bandit · helm lint

backend + frontend
built once

fixable HIGH/CRITICAL
fails the build

the scanned bytes
tag = commit SHA

kind + helm install
smoke: 201 then 409

SHA committed to
gitops/ → Argo CD

11/11
jobs green on the first run

3m 01s
from push to promoted image

0
registry passwords stored
(GITHUB_TOKEN)

`ghcr.io/rajprakash681/clinicflow-backend:8baee138a443…`

Why SHA tags
Every image traces back to its commit; nothing is ever "latest".

Scan what you ship
The image is saved after the scan and pushed unchanged, so no rebuild can slip in.

DevSecOps: the gate blocked a real CVE

Gitleaks
whole Git history, no credentials

Bandit + npm audit
Python SAST, dependency CVEs

Trivy
both images; fixable HIGH/CRITICAL fails

Runtime
non-root, read-only FS, no capabilities

Admission
restricted Pod Security on both namespaces

CVE-2026-4775

HIGH · libtiff 4.7.1 → fixed in 4.7.2 · code execution or DoS via a crafted TIFF

- ClinicFlow never reads TIFF: libtiff came in through nginx's image-filter module → libgd
- The image already ran apk upgrade, but Docker reused a stale cached layer
- Fix: delete the four nginx modules the app never loads

exit 1 → 0
Trivy gate, before vs after

70 → 31
packages in the frontend image

Infrastructure as code: AWS VPC + EKS



23
resources in the plan

0
left after destroy

- Plain resources, no modules: every object is visible in main.tf
- Credentials from aws configure; only tfvars.example is committed
- Cost: one NAT for both AZs; enable_nat_gateway=false if no spare EIP

Honest scope: no AWS account, so init / plan / apply / destroy ran against the moto emulator. Same code; real AWS needs only credentials.

Kubernetes, Helm and GitOps

Deployments

backend + frontend, 2 replicas each, probes, maxUnavailable 0

Services + Ingress

ClusterIP; clinicflow.local: / → frontend, /api → backend

StatefulSet + PVC

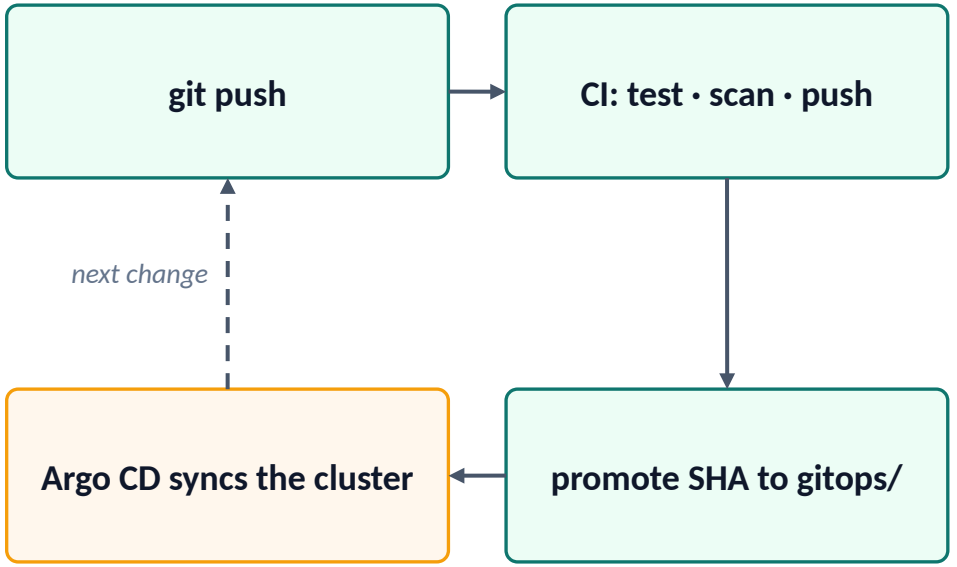
PostgreSQL, UID 70, 1 Gi

HPA + PDBs

backend 2–6 at 60% CPU; one pod per tier may be evicted

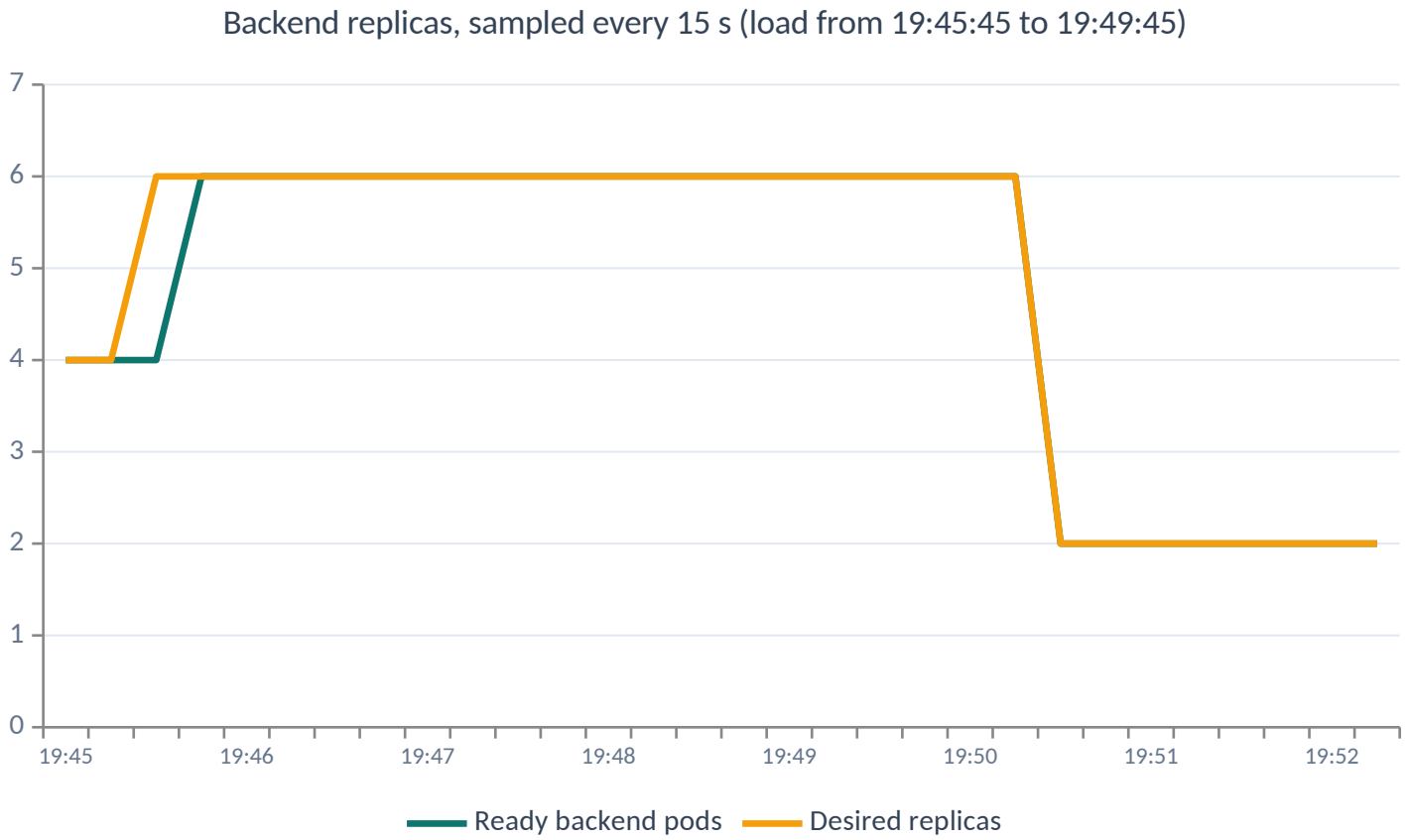
Guard rail

no image tag = no render: every deploy names a SHA



Staging: helm upgrade --install (helm list)
Production: Argo CD auto-sync · prune · self-heal
Nobody runs kubectl apply on production.

Proved under load: autoscaling and migrations



4 → 6

replicas within 30 s; CPU about 720% of target

→ 2

about 75 s after the load stopped

Migration race test

3 pods, empty database, same moment: one migrates, two wait on the advisory lock, all exit 0.

Observability: from HTTP latency to no-show rate

- /metrics: requests and latency per route, buckets from 5 ms
- Clinic gauges read from the DB at scrape time: today by status, patients waiting, 30-day no-show ratio
- ServiceMonitor → Prometheus scrapes every pod (6/6 UP under load)
- 7 alerts: backend down, 5xx, p95, HPA at max, no-show rate, waiting room, double-booking bursts
- 19-panel Grafana dashboard shipped in the chart as code



Key learnings

A gate is only as fresh as its cache

apk upgrade sat in a cached layer; removing the dependency was the real fix.

Git is the deploy interface

CI writes the SHA, Argo CD applies it. Argo renders Helm without a Helm release, so staging is the one that shows up in helm list.

One owner per field

The HPA owns replicas; Argo CD ignores them. The API owns opening hours; the UI reads them.

Check what the dashboard says

A stacked panel turned a few expected 409s into an apparent flood of errors.

Limitations and next steps

1

No real AWS run

Terraform ran on moto; the next step is a real apply with credentials, then destroy

2

No authentication

Patient data needs OIDC at the Ingress and TLS before any real use

3

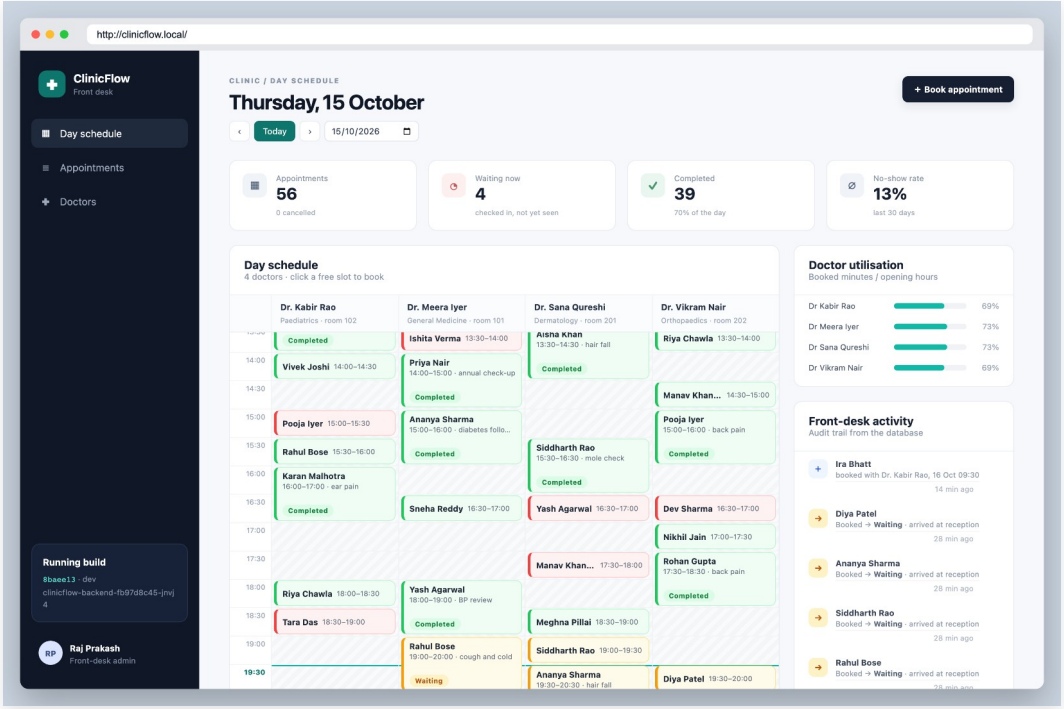
Single PostgreSQL

No backups; production values expect Amazon RDS

4

Polling GitOps

Argo CD checks GitHub every 60 s; a webhook needs a reachable cluster



Running today: kind cluster, Argo CD-managed, build shown in the sidebar

Thank you

Repository	github.com/RajPrakash681/devsecops-python
Demo video	demo/ClinicFlow-demo.mp4
Evidence	README.md, 34 screenshots from real runs
Pipeline	GitHub Actions, CI/CD - ClinicFlow

Raj Prakash · 24BCS10328